RAG (Retrieval-Augmented Generation)

Foundation Model 的限制

單純建立在 foundation model 之上的產品雖然聰明，但也有明顯的缺陷，因為 foundation model 本身存在幾個根本性的限制。

知識截止日期（Knowledge Cutoff）

當你問現在的模型最近發生的事，比如上週的 NBA 比賽結果，或者最新款 iPhone 的某個功能怎麼用，它可能會充滿自信地給你過時或完全捏造的資訊，也就是我們前面提到的 hallucination。

模型是在包含多年人類知識和創作成果的龐大資料集上訓練的，這些資料來自 code repository、書籍、網站、對話、科學論文等等。但模型一旦訓練完畢，這些資料就被凍結在某個特定的時間點，也就是「截止日（cutoff）」。這個截止日製造了一個知識缺口，讓模型在被問到近期發展時，產生聽起來合理但實際上是錯的回應。

缺乏特定領域的深度知識

Foundation model 的知識廣博，但在某些專業領域可能缺乏足夠的深度。高品質的資料集不一定以公開形式存在，不一定是因為它們是機密，而是因為它們高度專業化。想像一個醫療模型，它了解解剖學、疾病和手術技術，但在罕見遺傳疾病和最新療法上卻力不從心。這些資料也許公開存在，但出現的頻率可能不足以讓模型正確地學習，而且訓練過程中還需要專家級的知識來幫助模型理解這些資訊的脈絡。

這個限制會導致模型的回應不完整或不相關。

缺乏私有或專有資料（Private or Proprietary Data）

對於通用型的公開模型而言，你的資料（你的公司資料）並不存在於公開領域，在訓練過程中也無從取得。這意味著模型不知道你的業務細節，不管是公司內部流程和政策、人事資料、電子郵件通訊，還是公司的商業機密。這樣的設計是合理的，這些資料被納入訓練，任何使用這個模型的人都可能存取到你公司的私有和專有資料。

同樣地，這個限制會導致不完整或不相關的回應，讓模型在你的特定業務需求上用途大打折扣。

失去用戶信任

bilimoa.org

模型通常無法為特定的回應引用來源。沒有引用或參考資料，用戶要麼盲目相信，要麼得自己去驗證。考慮到模型是在大量公開資料上訓練的，產生的回應很有可能來自不具權威性的來源。

當模型不斷產生不準確、不相關、沒有用的資訊，用戶就會對模型本身失去信任，即使這種行為本來就是 foundation model 運作方式的一部分。

輸出生成是機率性的（Probabilistic）

Hallucination 通常是上述那些限制的症狀。但除此之外，模型訓練的資料集本身也可能包含矛盾、錯誤和模糊的資料（以及正確的資料混在一起）。因此，模型對所有可能的後續內容都分配了機率，包括錯的那些。加上 temperature 和 top-k 這類 sampling 的隨機性，以及用戶構建 prompt 的方式（也許太模糊，或包含誤導性的脈絡），模型可能選到錯誤的後續內容。結果就是輸出中出現了 hallucination。

此外，模型並不總是能區分「我知道什麼」和「我不知道什麼」，即使在不完整、不準確或不相關的情況下，也顯得十分自信。Hallucination 可能產生不理想的行為，甚至帶來危險。比如，一份不準確但極具說服力的醫療報告，可能導致危及生命的治療方式，或者根本不接受治療。

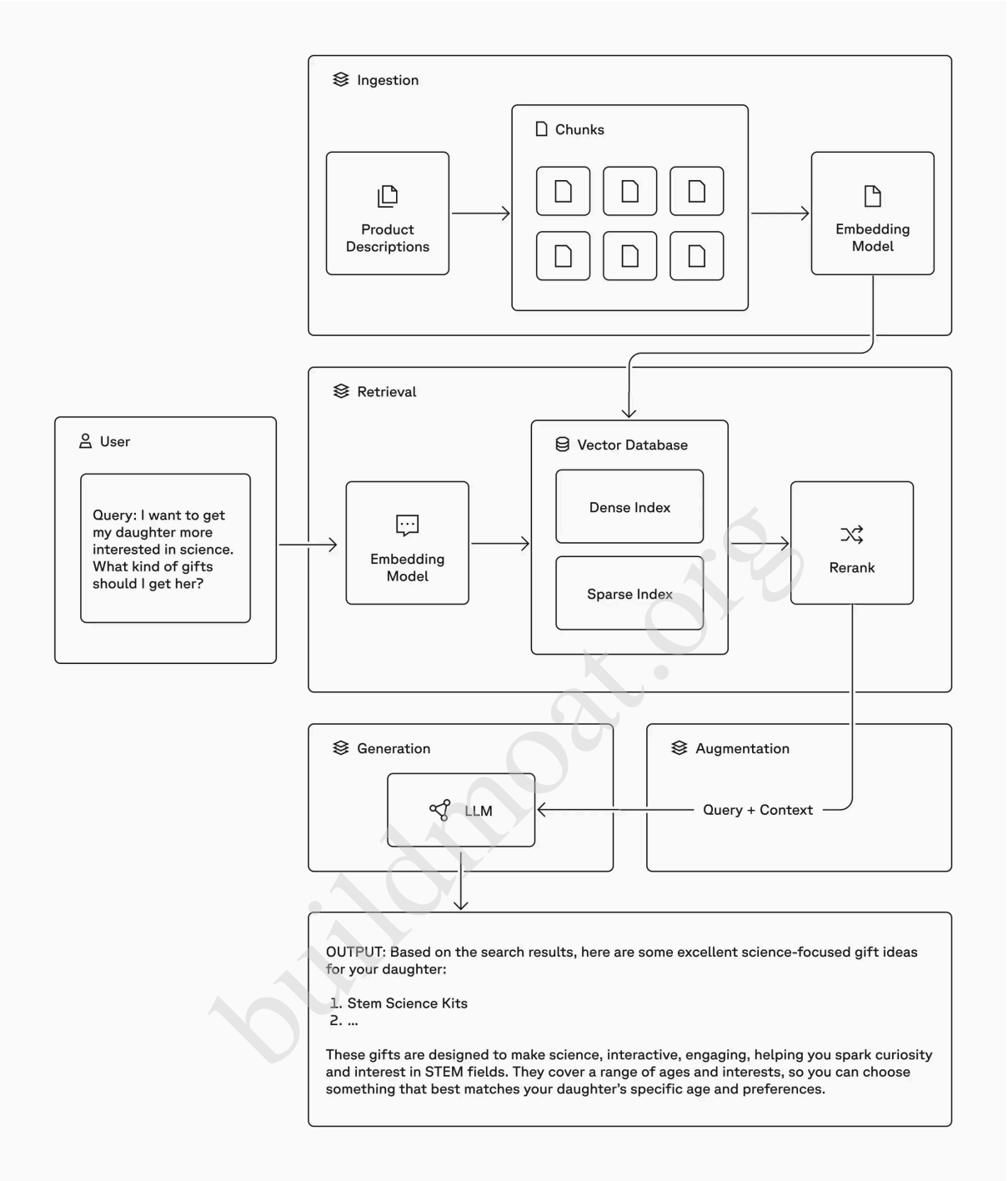
這些 foundation model 的限制，可能影響你的業務利潤，也可能侵蝕用戶的信任。而 retrieval-augmented generation 可以解決這些問題。

什麼是 Retrieval-Augmented Generation（RAG）？

Retrieval-augmented generation，也就是 RAG，是一種利用具有權威性的外部資料，來提升模型輸出的準確性、相關性和實用性的技術。它透過以下四個核心元件來實現，我們稍後會詳細介紹：

1. **Ingestion（資料攝取）**：像公司專有資料這樣的權威性資料，被載入資料來源，例如 Pinecone vector database
2. **Retrieval（檢索）**：根據用戶的 query，從外部資料來源檢索相關資料
3. **Augmentation（增強）**：把檢索到的資料和用戶的 query 組合成一個 prompt，為模型的生成步驟提供 context
4. **Generation（生成）**：模型根據增強後的 prompt 生成輸出，利用 context 產生更準確、更相關的回應

wildmoat.org



透過把外部資料來源中的相關資料和用戶 query 結合，作為生成步驟的 context 提供給模型，模型就會利用它來生成更準確、更相關的輸出。

RAG 帶來以下好處：

- **存取即時資料以及專有或特定領域的資料**：引入和你的情況相關的知識，比如當前事件、新聞、社群媒體、客戶資料、專有資料
- **建立信任**：更相關、更準確的結果更容易獲得信任，引用來源讓人工審查成為可能

- **更強的控制能力**：控制使用哪些資料來源、即時資料存取、資料授權、guardrail / 安全性 / 合規性、可追溯性與來源引用、retrieval 策略、成本，以及獨立調整每個元件
- **比替代方案更具成本效益**：相比自己訓練或重新訓練模型、fine-tuning，或把大量資訊塞進 context window，RAG 更划算，因為foundation model 的製作成本高昂，需要專業知識，fine-tuning 亦然；發送給模型的 context 越大，成本越高

RAG 在 Agentic Workflow 中的角色

傳統的 RAG 方式相對簡單，通常是一個 vector database 加上一次帶有 context 的 prompt 送給模型來生成輸出。隨著 AI agent 的興起，agent 現在成為了核心 RAG 元件的 orchestrator，用來：

- 構建更有效的 query
- 存取額外的 retrieval 工具
- 評估檢索到的 context 的準確性和相關性
- 應用推理來驗證檢索到的資訊，決定相信或捨棄它

這些操作可以由一個或多個 agent 作為更大型、迭代式計畫的一部分來執行。Agent 作為 RAG 的 orchestrator，帶來了更多 review、修改 query、推理或驗證 context 的機會，讓它們能做出更好的決策、採取更明智的行動，並生成更準確、更相關的輸出。

buildnoat.org

Retrieval-Augmented Generation 是怎麼運作的？

RAG 透過依賴具有權威性的資料來源，如專有的、特定領域的資料和即時資訊來為 LLM 的輸出帶來準確性和相關性。但在深入探討它怎麼做到之前，先問兩個問題：你真的需要 RAG 嗎？你怎麼知道它在正常運作？

這就是 **ground truth evaluation（基準評估）** 的用武之地。要正確部署任何應用，你需要知道它什麼時候在正常工作。AI 應用也不例外，因此，確定一組 query 和它們的預期答案，對於判斷應用是否正常工作至關重要。維護這個評估集，對於知道哪裡需要改進，以及那些改進是否真的有效，同樣重要。RAG 本身只是一個優化手段，還有其他手段如 query rewriting、chunk expansion、knowledge graph 等等。

有了好的 baseline，就可以開始實作 RAG 的四個主要元件。

第一步：Ingestion（資料攝取）

在簡單的傳統 RAG 中，你會從 vector database（如 Pinecone）中檢索資料，使用 semantic search 來找出用戶 query 的真實含義，並檢索相關資訊，而不是單純做關鍵字比對。我們以 Pinecone 為例，但這個概念適用於所有 vector database。

但在能夠檢索資料之前，你必須先把資料 ingest 進去。以下是把資料載入 database 的步驟：

分塊（Chunking）

在 ingestion 步驟中，你要把你的權威性資料以 vector 的形式載入 Pinecone。你的資料可能是結構化或非結構化的，可能是文字、PDF、電子郵件、內部 wiki，或 database。清洗資料之後，你可能需要把它分塊（chunk），也就是把每份資料或文件切成更小的片段。根據你的資料類型、用戶的 query 類型，以及結果在應用中的使用方式，你需要選擇一個合適的 chunking 策略。

建立 Vector Embedding

然後，使用一個 embedding model，對每個 chunk 做 embedding，並載入 vector database。Embedding model 是一種特殊的 LLM，它把資料 chunk 轉換成 vector embedding，也就是資料含義的數值表示。這讓電腦可以根據儲存資料的 vector 表示，搜尋到語義相似的項目。

載入 Vector Database有了 vector 之後，你把它們載入 vector database。這個 ingestion 步驟通常是 offline 進行的，獨立於你的應用和用戶的工作流程。不過，如果你的資料有變動，例如商品庫存更新，你可以即時更新 index，為用戶提供最新的資訊。

第二步：Retrieval（檢索）

一個簡單的 retrieval 方式只會用到 semantic search。但透過 **hybrid search**，也就是結合 semantic search（使用 dense vector）和 lexical search（使用 sparse vector），可以進一步提升 retrieval 的結果。這在用戶不總是用相同語言談論同一個主題（semantic search 擅長處理這種情況），或者他們使用內部的、特定領域的語言（lexical 或 keyword search 擅長處理這種情況），比如縮寫、產品名稱或團隊名稱時，特別有用。

在 retrieval 過程中，我們會從用戶的 query 建立一個 vector embedding，用來對 database 中的 vector 做搜尋。在 hybrid search 中，你可以查詢一個單一的 hybrid index，或者同時查詢一個 dense index 和一個 sparse index。然後合併並去重結果，使用 reranking model 根據統一的 relevance score 重新排序，最後回傳最相關的結果。

第三步：Augmentation（增強）

buildmeat.org

有了 retrieval 步驟中最相關的結果，你現在要把搜尋結果和用戶的 query 組合成一個增強後的 prompt，發送給 LLM。這就是魔法發生的地方。

一個增強後的 prompt 可能長這樣：

```
QUESTION:
<用戶的問題>

CONTEXT:
<用來作為 context 的搜尋結果>

Using the CONTEXT provided, answer the QUESTION. Keep your
answer grounded in
the facts of the CONTEXT. If the CONTEXT doesn't contain th
e answer to the
QUESTION, say you don't know.
```

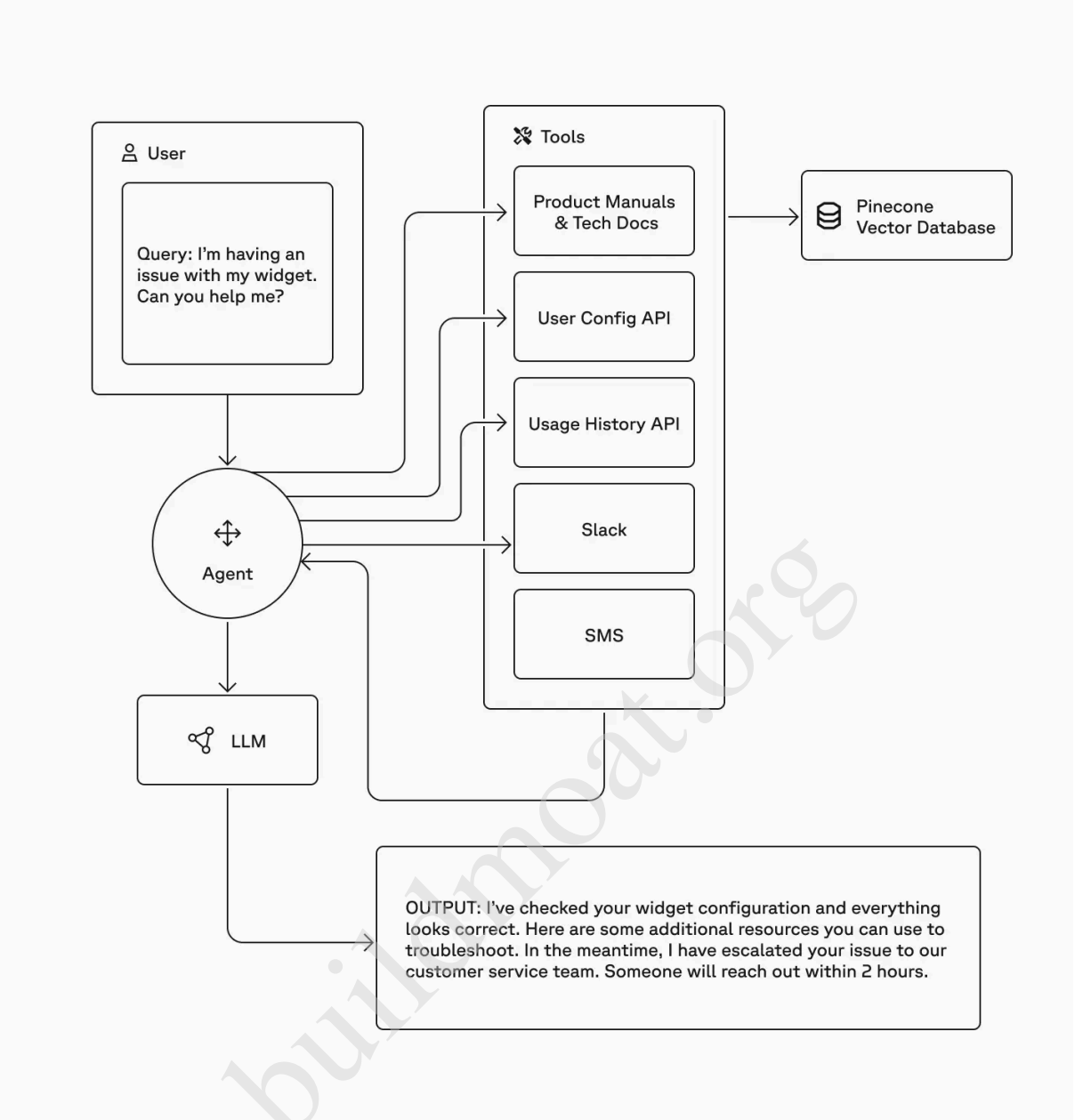
透過把搜尋結果和用戶的問題一起作為 context 發送給 LLM，你在鼓勵它在接下來的生成步驟中，優先使用搜尋結果中更準確、更相關的資訊。

第四步：Generation（生成）

使用增強後的 prompt，LLM 現在能存取到 vector database 中最相關的、紮實的事實，你的應用就能為用戶提供準確的答案，大幅降低 hallucination 的可能性。

但 RAG 已經不再只是搜尋正確的資訊來輔助模型回應了。在 agentic RAG 中，重點在於決定要問哪些問題、使用哪些工具、什麼時候使用它們，然後聚合結果來紮實地回答問題。

buildmoat.org



在這個簡化版本中，LLM 就是 agent，它自行決定要使用哪些 retrieval 工具、何時使用，以及如何查詢這些工具。

總結

Retrieval-augmented generation 已經從一個流行詞演變成 AI 應用不可或缺的基礎。它把 foundation model 的廣泛能力，與你公司的權威性和專有知識融合在一起。隨著 AI agent 處理越來越複雜的使用場景，從支援技術人員維護複雜製造設備，到大規模交付特定領域的 agent，RAG 不只是在 2025 年依然相關，它是構建準確、相關、負責任的 AI 應用的關鍵，遠超過單純的資訊檢索。

隨著 AI agent 變得更加自主並處理更複雜的 workflow，它們需要透過 RAG，把推理紮根在你的私有和特定領域資料上。問題已經不再是「要不要實作 RAG」，而是「如何針對你的獨特使用場景和資料需求，最有效地設計它的架構」。

buildmoat.org